

SOFTWARE ENGINEERING PROJECT

PARKEASE

Parking Management & Reservation System

PROTOTYPE DOCUMENT



Prepared by:

Soham Kumar (1024030775)
Yuvraj Bhardwaj (1024030771)
Tanishk Batra (1024030769)

Department: Computer Engineering
Thapar Institute of Engineering and Technology

September 2026

Contents

1	Prototype Overview	3
2	Objectives	3
3	Prototype Workflow	3
3.1	User Registration and Login	3
3.2	Vehicle Profile	4
3.3	Parking Facility Search	4
3.4	Occupancy and Available Slots	4
3.5	Reservation	4
3.6	Payment	4
3.7	Confirmation and QR Code	4
3.8	Booking History	4
4	Technology Stack	4
4.1	Frontend Technology Stack	4
4.2	Backend Technology Stack	5
5	Frontend Features Implemented	5
6	Backend API	5
7	MongoDB Database Schema	6
8	User Workflow	6
9	What is Working in the Prototype	6
9.1	Complete Authentication	6
9.2	CRUD Operations	6
9.3	API Integration	6
9.4	Data Persistence	7
9.5	Responsive Design	7
9.6	Search Functionality	7
10	Prototype Demonstration Links	7

11 How to Test the Prototype	7
12 Next Phase — Roadmap	8
12.1 Phase 2	8
12.2 Phase 3	8
12.3 Phase 4	8
13 Conclusion	8

1 Prototype Overview

ParkEase is a parking management and reservation system prototype designed to provide users with a convenient way to discover parking facilities, view occupancy, manage vehicles, make reservations, complete payments, and access booking history.

The working prototype combines a React-based responsive web frontend with a Node.js and Express REST API backend and MongoDB database integration.

The basic prototype flow is:

**Authentication → Vehicles → Facilities → Search → Occupancy → Reservation →
Payment → Confirmation → Booking History**

The prototype demonstrates the core end-to-end parking workflow, from user registration and facility discovery through reservation and payment confirmation.

2 Objectives

The main objectives demonstrated by the ParkEase prototype are:

- Provide user registration and login functionality.
- Provide JWT-based secure authentication.
- Allow users to add and manage multiple vehicles.
- Allow users to search parking facilities by location or facility name.
- Display real-time occupancy and available parking slots.
- Allow users to make and manage parking reservations.
- Provide payment integration as part of the booking flow.
- Provide an interactive map view for parking facilities.
- Maintain booking history and ratings.
- Persist application data using MongoDB.

3 Prototype Workflow

3.1 User Registration and Login

The user begins by registering a new account or logging into an existing account. The prototype implements authentication through the backend using JWT-based authentication.

3.2 Vehicle Profile

After authentication, the user completes the vehicle profile. The system supports adding and managing multiple vehicles, with vehicle information stored in the database.

3.3 Parking Facility Search

The user can search for parking facilities using their location or the name of a parking facility. The frontend communicates with the backend REST API to retrieve facility information.

3.4 Occupancy and Available Slots

The prototype displays parking occupancy and available slots. Users can view the availability before proceeding with a reservation.

3.5 Reservation

The user selects the required floor and reserves a parking slot. Reservation information is submitted through the backend API and stored in MongoDB.

3.6 Payment

After making a reservation, the user proceeds to the payment stage. The working prototype includes payment integration using a mock payment flow.

3.7 Confirmation and QR Code

After payment, the prototype provides a confirmation containing a QR code. This provides the user with a confirmation of the completed booking.

3.8 Booking History

The user can view the reservation in the booking history after completing the booking process.

4 Technology Stack

4.1 Frontend Technology Stack

Technology	Role in Prototype
React	Component-based UI library
React Router	Client-side navigation
CSS/Tailwind	Styling and responsive design

Local StorageClient-side data persistence

4.2 Backend Technology Stack

Technology	Role in Prototype
Node.js	JavaScript runtime
Express	Web framework
MongoDB	NoSQL database
JWT Auth	Secure authentication
REST API	API architecture

5 Frontend Features Implemented

The following frontend features are implemented in the working prototype:

1. User Registration & Login
2. Add/Manage Multiple Vehicles
3. Search Parking Facilities by Location
4. Real-time Occupancy Display
5. Make & Manage Reservations
6. Payment Integration
7. Interactive Map View
8. Booking History & Ratings

6 Backend API

The prototype exposes REST API endpoints for authentication, facilities, vehicles, reservations, and payments.

Endpoint	Purpose
POST /auth/register	User registration
POST /auth/login	User login
GET /facilities	Get all facilities
GET /facilities/:id	Get facility details
GET /vehicles	Get user vehicles
POST /reservations	Create reservation
GET /reservations	Get user reservations
POST /payments	Process payment

7 MongoDB Database Schema

The working prototype uses MongoDB for storing and retrieving application data.

Collection	Fields Demonstrated in Prototype
Users	ID, Email, Password, Name, Phone, Created
Vehicles	ID, UserID, Plate, Type, Color, Year
Facilities	ID, Name, Location, Capacity, Rate
Slots	ID, FacilityID, FloorNo, Status, Type
Reservations	ID, UserID, SlotID, StartTime, EndTime
Payments	ID, ReservationID, Amount, Status, Date

8 User Workflow

The complete user workflow demonstrated by the prototype is:

1. Register / Login
2. Complete vehicle profile
3. Search parking by location or facility name
4. View real-time occupancy and available slots
5. Select floor and reserve a slot
6. Proceed to mock payment
7. Receive confirmation with QR code
8. View reservation in booking history

9 What is Working in the Prototype

The working prototype demonstrates the following capabilities:

9.1 Complete Authentication

JWT-based secure login and registration are implemented.

9.2 CRUD Operations

Create, read, update, and delete operations are implemented for the prototype entities.

9.3 API Integration

The frontend communicates with the backend through REST APIs.

9.4 Data Persistence

MongoDB is used to store and retrieve application data.

9.5 Responsive Design

The frontend is designed to work on desktop and mobile browsers.

9.6 Search Functionality

Parking facilities can be filtered according to location and availability.

10 Prototype Demonstration Links

Prototype Landing Page:

<https://parkeaselandingpageforprototypeexplai.netlify.app/>

Vercel Deployed Link:

<https://parkease-bbmr-ma9oqdzsn-yuvrajbhardwajs-projects.vercel.app/>

GitHub Repository:

Yuvrajbhardwaj11/software-engineeringProject

The repository contains:

- `/client` → React frontend
- `/server` → Node.js + Express backend

11 How to Test the Prototype

The prototype can be tested using the following procedure:

1. Clone the repository.
2. Install dependencies in the `/client` and `/server` directories.
3. Start MongoDB locally or use a cloud database.
4. Run the backend using `cd server && npm start` on port 5000.
5. Run the frontend using `cd client && npm start` on localhost:3000.
6. Register a new account and start the parking booking workflow.

12 Next Phase — Roadmap

The prototype presentation identifies the following future development phases:

12.1 Phase 2

- ANPR Camera Integration
- Real-time slot allocation
- Mobile App using React Native

12.2 Phase 3

- Dynamic Pricing Algorithm
- Wallet system for locals
- Advanced Analytics Dashboard

12.3 Phase 4

- Multi-city expansion
- IoT sensor integration
- ML-based occupancy prediction

13 Conclusion

The ParkEase working prototype demonstrates a complete parking reservation workflow supported by a React frontend, Node.js and Express backend, and MongoDB database. It provides core functionality for authentication, vehicle management, parking facility search, occupancy viewing, reservations, payment, confirmation, and booking history.

The prototype therefore provides a functional foundation for the continued development of the ParkEase parking management system, with future phases focusing on ANPR integration, real-time slot allocation, mobile support, dynamic pricing, analytics, IoT integration, and ML-based occupancy prediction.